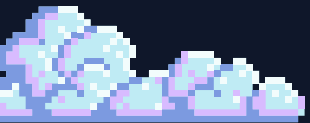


Smart Home Security System

A Software Simulation Built Using Logic Gates (AND / OR / NOT Circuits)

Presented by:

- B. Lahari - 2620030596
- Keerthi Sri - 2620030621
- Tarun Teja - 2620030594
- Ashish Reddy - 2620030610



What We'll Cover

Problem & Objectives

- > Why smart homes need rule-based intrusion logic

System Architecture

- > Sensors, gate circuits, controller, alerts

Logic Gate Fundamentals

- > AND / OR / NOT as the rule-building blocks

Gate-Based Rule Design

- > Turning security rules into logic circuits

Simulation & Testing

- > Java simulation + Logisim circuit verification

Advantages & Future Scope

- > Why this design approach, and where it goes next

The Problem !

Traditional home alarm panels react to a **single trigger at a time** — one door sensor, one motion sensor — with rigid, hard-coded wiring that's difficult to modify or reason about.

- Real intrusions depend on combinations of conditions, not isolated events
- Rules like “motion + armed + night-time” need logical composition
- A software simulation lets rules be designed, tested, and explained visually — before any hardware is built

Project Objectives



Model sensors as boolean signals

Represent doors, windows, motion and smoke sensors as simple true/false states.



Build a reusable logic-gate engine

Implement AND, OR and NOT gates as composable software objects.



Express security rules as circuits

Combine gates into named rules such as perimeter breach or night-time motion.



Simulate and verify behaviour

Run the system through a CLI and a Logisim circuit to confirm correct alerts.

System Architecture

Four stages, each mapped to a Java package in the simulation



Whenever any sensor fires, the controller re-evaluates every gate circuit; any rule that evaluates true raises a prioritized alert.

PerimeterBreach

InteriorMotion

NightMotionHome

SmokeDetected

Logic Gate Fundamentals

The three gates every security rule in this project is built from



AND

Output is TRUE only when every input is TRUE.

`Motion AND ArmedAway`



OR

Output is TRUE when at least one input is TRUE.

`FrontDoor OR BackDoor OR Window`



NOT

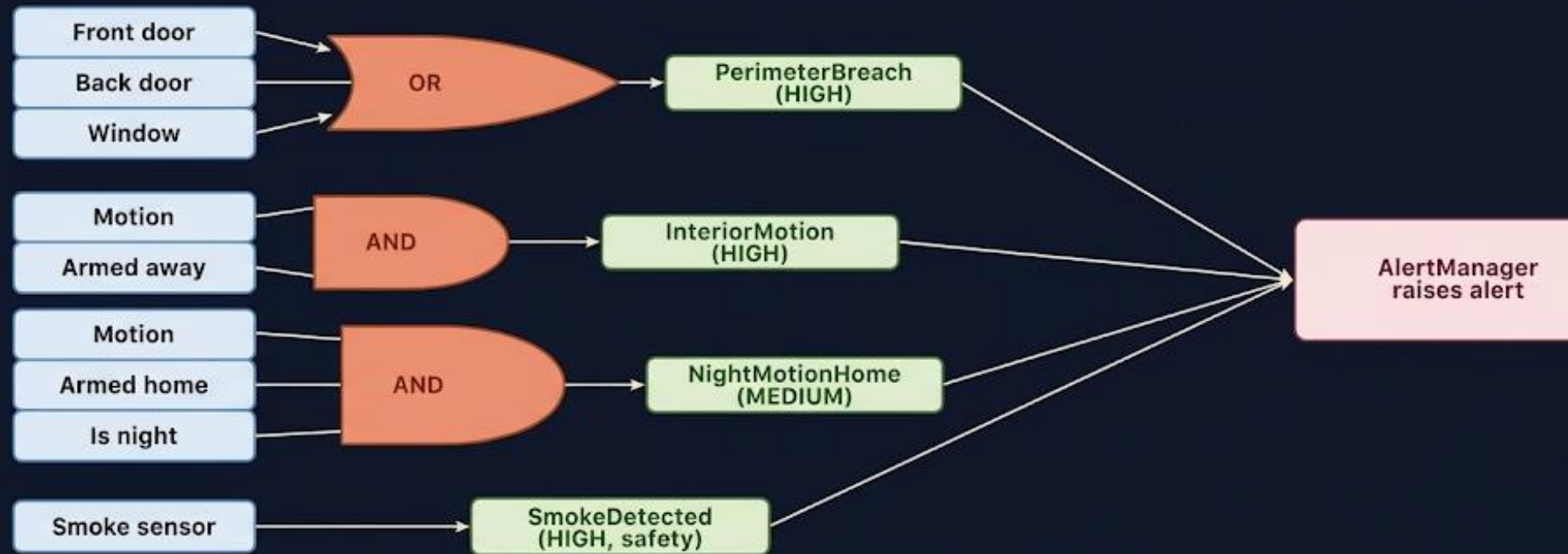
Flips the input — TRUE becomes FALSE.

`NOT Silenced`

Turning Rules Into Circuits

Each security rule is a small circuit of nested gates — sensors and system conditions feed in, a named alert comes out

Smart Home Security System — Rules Built From Logic Gates



Verifying a Rule: Truth Table

Rule: `InteriorMotion = Motion AND ArmedAway`

Motion	ArmedAway	Alert Raised?
0	0	No
0	1	No
1	0	No
1	1	Yes

Reading it

Only the last row — both inputs TRUE — fires the alert. That's exactly what the AND gate enforces: motion alone (armed or not) never triggers InteriorMotion unless the system is armed away.

Every rule in the project can be checked this same way before it's ever run in code or Logisim — which is the real value of designing with gates.

Simulation & Testing

The same gate logic was verified two ways

Java Software Simulation

- Gate interface with AndGate / OrGate / NotGate implementations
- SecurityController re-evaluates all rules on every sensor event
- Interactive CLI to arm/disarm, trigger sensors, view alert log
- Alerts logged to console and to a text file

Result: motion during ARMED_HOME stayed silent by day, and correctly fired once night-mode was toggled on.

Logisim Circuit Verification

- Each rule rebuilt as real AND / OR gates with input pins
- Sensors represented as clickable toggle pins
- Output pin lights up the instant a rule condition is met
- Confirms the software logic matches true digital-circuit behaviour

Result: all four rules verified pin-by-pin, matching the Java simulation's alert output exactly.

Why Design With Logic Gates?



Modular

Every rule is an independent, nameable circuit — easy to add, remove, or adjust.



Explainable

A truth table or circuit diagram proves correctness before a single line runs.



Composable

Gates nest inside gates, so complex rules build cleanly from simple ones.



Hardware-ready

The exact same logic maps onto real ICs or a Logisim circuit with no redesign.

Future Scope



Real sensors

Replace simulated inputs with ESP32/Arduino hardware sensors.



More gate types

Add TimedGate and XorGate for richer, time-aware rules.



Live notifications

Wire the AlertObserver to real SMS/email APIs.



Smarter detection

Layer in anomaly detection to flag faulty sensors vs. real intrusions.

Thank You

